

Festival

Nayra se na festivalu udeleži igre, kjer je glavna nagrada izlet v Laguno Colorada. V igri z žetoni kupuje kupone, pri čemer ji lahko vsak kupon prinese dodatne žetone za nove nakupe. Njen cilj je zbrati čim več kuponov.

Na začetku igre ima A žetonov. Obstaja N kuponov, oštevilčenih od 0 do $N - 1$. Nayra mora plačati $P[i]$ žetonov ($0 \leq i < N$), da kupi kupon i (in mora imeti vsaj $P[i]$ žetonov pred nakupom). Vsak kupon lahko kupi največ enkrat.

Poleg tega je vsakemu kuponu i ($0 \leq i < N$) dodeljen **tip**, označen s $T[i]$, ki je celo število **med 1 in 4, vključno**. Ko Nayra kupi kupon i , se število preostalih žetonov, ki jih ima, pomnoži s $T[i]$. Če ima Nayra na neki točki igre X žetonov, in kupi kupon i (kar zahteva $X \geq P[i]$), bo po nakupu imela $(X - P[i]) \cdot T[i]$ žetonov.

Vaša naloga je določiti, katere kupone naj Nayra kupi in v kakšnem vrstnem redu, da bo na koncu imela največje možno število **kuponov**. Če obstaja več zaporedij nakupov, ki to omogočajo, lahko sporočite katerega koli izmed njih.

Podrobnosti implementacije

Implementirajte naslednjo funkcijo:

```
std::vector<int> max_coupons(int A, std::vector<int> P,
                             std::vector<int> T)
```

- A : začetno število žetonov, ki jih ima Nayra.
- P : polje dolžine N , ki določa cene kuponov.
- T : polje dolžine N , ki določa tipe kuponov.
- Za vsak testni primer se funkcijo pokliče natanko enkrat.

Funkcija naj vrne polje R , ki določa Nayrine nakupe na naslednji način:

- Dolžina R naj bo največje možno število kuponov, ki jih lahko Nayra kupi.
- Elementi polja so indeksi kuponov, ki jih mora kupiti, v kronološkem vrstnem redu. To pomeni, da najprej kupi kupon $R[0]$, nato kupi kupon $R[1]$ in tako naprej.
- Vsi elementi R morajo biti različni.

Če ni možno kupiti nobenega kupona, mora R biti prazno polje.

Omejitve

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ za vsak i , kjer $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ za vsak i , kjer $0 \leq i < N$.

Podnaloge

Podnaloga	Točke	Dodatne omejitve
1	5	$T[i] = 1$ za vsak i , kjer $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ za vsak i , kjer $0 \leq i < N$.
3	12	$T[i] \leq 2$ za vsak i , kjer $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra lahko kupi vseh N kuponov (v nekem vrstnem redu).
6	16	$(A - P[i]) \cdot T[i] < A$ za vsak i , kjer $0 \leq i < N$.
7	18	Ni dodatnih omejitev.

Primeri

1. primer

Razmislite o naslednjem klicu.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayra ima na začetku $A = 13$ žetonov. Kupi lahko 3 kupone v vrstnem redu, prikazanem spodaj:

Kupon	Cena	Tip	Število žetonov po nakupu
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

V tem primeru Nayra ne more kupiti več kot 3 kupone, in zgoraj opisano zaporedje nakupov je edini način, da jih kupi 3. Zato mora funkcija vrniti $[2, 3, 0]$.

2. primer

Razmislite o naslednjem klicu.

```
max_coupons(9, [6, 5], [2, 3])
```

V tem primeru lahko Nayra kupi oba kupona, v kakršnem koli vrstnem redu. Zato mora funkcija vrniti $[0, 1]$ ali $[1, 0]$.

3. primer

Razmislite o naslednjem klicu.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

V tem primeru ima Nayra en žeton, kar ni dovolj za nakup nobenega kupona. Zato mora funkcija vrniti $[]$ (prazno polje).

Vzorčni ocenjevalnik

Oblika vhoda:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Oblika izhoda:

```
S
R[0] R[1] ... R[S-1]
```

Pri čemer je S dolžina polja R , ki ga vrne `max_coupons`.